UNIVERSIDAD TECNOLÓGICA DEL VALLE DE TOLUCA

INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN E INNOVACIÓN DIGITAL CON TSU EN DESARROLLO DE SOFTWARE MULTIPLATAFORMA

ASIGNATURA:

FUNDAMENTOS DE INTELIGENCIA ARTIFICIAL

NOMBRE DEL DOCENTE:

MGTI. HÉCTOR VELÁZQUEZ ESTRADA

“Manual técnico del Sistema LumaIA”

PRESENTA:

222410869 – JUÁREZ GONZÁLEZ IRAIS
222410849 – MENDOZA SANCHEZ ZUHEIDI
222410893 - NOYOLA SANABRIA JAREDZI AIDE
222411200 – ROBLEDO PADILLA DORIAN BRISA
222411167- SALINAS ARANDA OSVALDO

GRUPO: ITIID-D71 **EQUIPO:**

SANTA MARÍA ATARASQUILLO, LERMA ESTADO DE MÉXICO.28 SEPTIEMBRE 2026

Versión	Fecha	Descripción	Autor(es)
1.0	28 de septiembre de 2026	Manual técnico para el desarrollo del sistema multiplataforma “LumaIA”, estructurado bajo el estándar IEEE 830. Incluye la arquitectura general del sistema, modelo relacional de base de datos, diagramas de arquitectura (componentes, servicios, despliegue y clases), así como la guía de instalación, configuración, despliegue y resolución de problemas técnicos comunes.	Mendoza Sánchez Zuheidi Noyola Sanabria Jaredtzi Aide Salinas Aranda Osvaldo Juárez Gonzáles Irais Robledo Padilla Dorian Brisa

I. DECLARACIÓN DE DERECHOS DE AUTOR

© 2026 Jaredtzi Aide Noyola Sanabria, Zuheidi Mendoza Sánchez, Osvaldo Salinas Aranda, Irais Juárez González y Dorian Brisa Robledo Padilla.

Todos los derechos reservados.

El presente manual ha sido elaborado por Jaredtzi Aide Noyola Sanabria, Zuheidi Mendoza Sánchez, Osvaldo Salinas Aranda, Irais Juárez González y Dorian Brisa Padilla Robledo con fines educativos y académicos, protegidos conforme a lo establecido en la Ley Federal del Derecho de Autor (LFDA) y los tratados internacionales en la materia. La estructura, redacción, organización y contenido desarrollado específicamente para este manual son de autoría propia por parte de sus creadores, encontrándose legalmente registrados y protegidos por los artículos 11, 13, 18 y demás relativos de la LFDA. Queda estrictamente prohibida la reproducción, modificación, distribución, comercialización o publicación total o parcial de este documento con fines distintos a los académicos sin la autorización previa y por escrito de sus autores, bajo las sanciones civiles y penales previstas por la ley por la violación a los derechos patrimoniales y morales de autor. Los nombres, logotipos, programas, lenguajes de programación, sistemas de bases de datos y demás recursos tecnológicos pertenecientes a terceros mencionados en este manual conservan sus respectivos derechos de autor y propiedad intelectual. Su utilización dentro de este documento se realiza en apego al derecho de cita e ilustración de la ciencia y la educación (conforme al artículo 148 de la Ley Federal del Derecho de Autor), teniendo únicamente fines educativos, informativos y de referencia.

Contenido

I. DECLARACIÓN DE DERECHOS DE AUTOR 1
II. OBJETIVO 3
III. ALCANCES 3
IV. TÉRMINOS Y DEFINICIONES 3
V. Marco PEAS para el Sistema LumalA 4
VI. MODELO DE REQUERIMIENTOS (Reglas de negocio implementadas dentro del sistema) . 7
Historias de usuario 8
Documento de requerimientos 8
Acumulado del producto (Product backlog en la herramienta de Gestión de Proyectos de Desarrollo de Software) 9
Atributos de calidad 9
Preguntas de negocio 9
VII. PREREQUISITOS (descripción de los requerimientos mínimos de hardware o software para la instalación y puesta en funcionamiento del sistema) 10
Requerimientos de hardware (Infraestructura del servidor) 10
Requerimientos de Hardware (Cliente - Dispositivo Móvil) 11
Requerimientos Mínimos de Software (Sistemas y Servidores) 11
Lenguajes de Programación Utilizados 11
VIII. FRAMEWORKS Y ESTÁNDARES DE DESARROLLO UTILIZADOS 12
Librerías, Frameworks, Controladores o Plugins 12
Estándares de Codificación y Patrones de Diseño 12
Estándares de Identidad Corporativa y Look & Feel 12
Puertos de Comunicación 13
Protocolos de Seguridad 13
Interfaces con Otros Sistemas (Integraciones) 13
IX. MODELO DE DATOS 14
X. FUNCIONALIDAD Y SERVICIOS OFRECIDOS 18
XI. DESPLIEGUE Y CONFIGURACIÓN DE LOS COMPONENTES QUE CONFORMAN EL SISTEMA DE INFORMACIÓN 18
XII. RESOLUCIÓN DE PROBLEMAS 19
XIII. REFERENCIAS 20
XIV. CONCLUSIONES 20

II. OBJETIVO

Proporcionar una guía técnica integral y detallada sobre la arquitectura, diseño, configuración y despliegue del sistema multiplataforma "LumaIA". Este manual tiene como fin documentar el modelo de datos, la estructura de componentes y los estándares de desarrollo utilizados, asegurando que cualquier desarrollador o ingeniero de software pueda comprender, mantener o escalar el ecosistema (aplicación móvil, backend y motor lógico) de manera eficiente.

III. ALCANCES

El presente manual técnico abarca exclusivamente el diseño interno, la arquitectura lógica, los requerimientos de infraestructura y los diagramas estructurales (UML y bases de datos) del sistema LumaIA. Está dirigido a equipos de desarrollo, administradores de bases de datos y evaluadores técnicos. No incluye manuales de capacitación para el usuario final (Adulto Mayor o Cuidador), lo cual se documenta por separado en el Manual de Usuario.

IV. TÉRMINOS Y DEFINICIONES

- Adulto Mayor: Usuario principal del sistema es la persona que interactúa con la interfaz adaptada para organizar su medicación y agenda.
- API REST: Interfaz de Programación de Aplicaciones basada en el protocolo HTTP, utilizada para la comunicación bidireccional entre la aplicación móvil (frontend) y el servidor (backend).
- Caché Local: Mecanismo de almacenamiento temporal en el dispositivo móvil que garantiza la disponibilidad de la agenda diaria y contactos de emergencia sin conexión a internet (modo offline).
- Cron Job: Tarea programada en el servidor que se ejecuta automáticamente en intervalos definidos (utilizada en LumaIA para ejecutar el Hard Delete de datos tras 30 días).
- Cuidador: Usuario secundario (familiar o red de apoyo) con permisos lógicos vinculados a un Adulto Mayor, encargado de la gestión remota y recepción de notificaciones de emergencia.
- Hard Delete: Eliminación permanente e irrecuperable de un registro en la base de datos.
- Motor de Inferencia: Módulo lógico basado en programación proposicional, encargado de evaluar el contexto del usuario en la base de datos para responder dudas en lenguaje natural.
- Mobile-First: Estrategia de diseño y desarrollo tecnológico enfocada primordialmente en el

correcto funcionamiento y visualización en pantallas de dispositivos móviles.

- Push / Notificación Push: Alerta automatizada enviada desde el servidor que aparece directamente en la pantalla del dispositivo móvil del Cuidador, incluso si la aplicación está cerrada.
- Soft Delete: Borrado lógico en la base de datos. El registro no se elimina, sino que cambia su estado a "inactivo" para permitir su recuperación durante el periodo de gracia de 30 días.
- Speech-to-Text / Text-to-Speech (STT/TTS): Tecnologías de hardware y software que permiten transformar la voz del usuario a texto procesable, y viceversa, facilitando la interacción con el Asistente Lógico.
- Token (JWT): Cadena de caracteres encriptada (JSON Web Token) utilizada para validar y persistir la sesión de un usuario de forma segura sin requerir el ingreso constante de credenciales.
- Touch Target: Área interactiva (botones, campos) en una interfaz táctil, diseñada con un tamaño ampliado para facilitar la pulsación por parte de usuarios con movilidad reducida.
- WCAG (Web Content Accessibility Guidelines): Conjunto de directrices de accesibilidad adoptadas para garantizar el contraste de color y legibilidad de la interfaz de LumaIA.

V. Marco PEAS para el Sistema LumaIA

El marco PEAS (Performance, Environment, Actuators, Sensors) define los agentes inteligentes del sistema (enfocados en el asistente conversacional, la gestión de rutinas y el sistema de alertas):

P (Medida de Rendimiento / Performance):	• Garantizar la puntualidad y precisión en los recordatorios de medicamentos y citas médicas para el adulto mayor. • Asegurar un tiempo de respuesta del motor lógico inferior a 2 segundos. • Cumplir de forma estricta con la restricción de no realizar diagnósticos médicos en ninguna circunstancia. • Maximizar la efectividad en el escalamiento de alertas al cuidador en un lapso máximo de 15 minutos ante la omisión de tomas críticas. • Acatar los estándares de accesibilidad WCAG AA (contraste mínimo de 4.5:1 y *Touch Targets* ampliados)

E (Entorno / Environment):	• Dispositivos móviles inteligentes (Android / iOS) operados bajo un enfoque *Mobile-First*. • Entorno de red híbrido con soporte para caché local (modo *offline* parcial para agenda y contactos de emergencia). • Infraestructura de backend basada en Node.js/NestJS y base de datos relacional/NoSQL. • Red de apoyo externa compuesta por el cuidador y servicios de notificaciones push (FCM / APNs)
A (Actuadores / Actuators):	• Módulo Text-to-Speech (TTS) para la síntesis de voz y reproducción auditiva de respuestas. • Envío automatizado de notificaciones Push a los dispositivos de los cuidadores. • Activación de alertas de emergencia y llamadas a través del Botón SOS. • Actualización de estados de tomas, registros médicos y borrados lógicos (*Soft Delete*) en la base de datos
S (Sensors / Sensors):	• Micrófono del dispositivo móvil para la captura de comandos de voz (*Speech-to-Text*). • Pantalla táctil para la interacción mediante botones grandes y de alto contraste (*Touch Targets*). • Reloj del sistema y tareas programadas (*Cron Jobs*) para la evaluación de tiempos de confirmación de medicamentos. • Tokens de autenticación (JWT) y validadores de estado de sesión.

Definición de Variables Proposicionales

Para el funcionamiento del motor de inferencia conversacional y de control de rutinas del sistema, se definen las siguientes variables proposicionales booleanas:

P: El recordatorio de un medicamento crítico ha sido emitido al usuario.

Q: El adulto mayor ha confirmado la toma del medicamento dentro del límite de 15 minutos.

R: El sistema genera y envía una notificación push de escalamiento al cuidador.

S: El usuario presiona el Botón SOS en la interfaz principal.

D: La premisa o consulta ingresada por voz contiene solicitudes de diagnóstico clínico o prescripción médica.

B: El motor lógico activa el bloqueo médico y restringe la respuesta automatizada (*Regla RD-01*).

Tablas de Verdad para las Reglas de Negocio

Tabla 1: Regla de Escalamiento por Omisión de Medicamento Crítico

Modela la regla de negocio donde si un medicamento crítico es emitido (P) y el adulto mayor no confirma su toma ($\neg Q$), el sistema escala la alerta al cuidador (R). Expresión lógica: $(P \wedge \neg Q) \rightarrow R$

P (Recordatorio emitido)	Q (Confirmación a tiempo)	$\neg Q$ (No confirmado)	$(P \wedge \neg Q)$ (Omisión detectada)	R (Notificación Push al Cuidador)
V	V	F	F	**V** (Operación normal / Tratamiento al día)
V	F	V	V	**V** (Alerta crítica escalada enviada)
F	V	F	F	**V** (Sin evento de omisión activo)
F	F	V	F	**V** (Sin eventos pendientes)

Tabla 2: Regla de Restricción Diagnóstica (Asistente Lógico)

Modela la directiva de seguridad ética del sistema LumalA donde, ante cualquier intento de consulta con fines de diagnóstico médico (D), el motor de inferencia debe activar obligatoriamente el bloqueo médico y rechazar la emisión de diagnósticos (B). Expresión lógica: $D \rightarrow B$

D (Entrada con solicitud de diagnóstico)	B (Activación de Bloqueo / Negativa de diagnóstico)	Implicación lógica ($D \rightarrow B$)
V	V	**V** (Acción correcta: se detecta la

		intención y se bloquea de forma segura)
V	F	**F** (Fallo crítico del sistema: permitiría un diagnóstico ilegal/no autorizado)
F	V	**V** (Comportamiento defensivo permitido)
F	F	**V** (Funcionamiento normal para consultas permitidas de agenda y horarios)

Tabla 3: Regla de Activación del Botón SOS

Modela la respuesta inmediata del sistema ante la pulsación del Botón SOS (S), la cual debe desencadenar obligatoriamente la alerta de emergencia hacia la red de apoyo (R). Expresión lógica: $S \rightarrow R$.

S (Presión del Botón SOS)	R (Disparo de Alerta de Emergencia al Cuidador)	Implicación lógica ($S \rightarrow R$)
V	V	**V** (Emergencia atendida, notificada y escalada)
V	F	**F** (Error crítico del sistema: botón inoperante)
F	V	**V** (Simulación o prueba manual de la red de apoyo)
F	F	**V** (Estado de reposo habitual del sistema)

VI. MODELO DE REQUERIMIENTOS (Reglas de negocio implementadas dentro del sistema)

El modelo de requerimientos de LumaIA define las reglas de negocio estructurales que el software ejecuta para garantizar la seguridad, accesibilidad y autonomía del adulto mayor, así como el monitoreo del cuidador, asegurando que la lógica de la plataforma cumpla con restricciones éticas (como la prohibición de diagnósticos médicos).

Historias de usuario

Para la construcción del sistema se empleó un enfoque ágil. A continuación, se presentan las historias de usuario clave que estructuran el desarrollo:

- HU-01 (Autenticación y roles): Como usuario, quiero registrarme en la plataforma y seleccionar mi rol (Adulto mayor o cuidador) para acceder a una interfaz adaptada a mis necesidades específicas.
- HU-02 (Vinculación de cuentas): Como cuidador, quiero vincular mi cuenta a la de un adulto mayor mediante un código PIN/QR temporal para poder monitorear y gestionar su agenda médica a distancia.
- HU-03 (Gestión de medicamentos): Como adulto mayor, quiero registrar mis medicamentos indicando dosis y horario a través de un formulario de alto contraste y botones grandes para no olvidar mis tratamientos.
- HU-04 (Interacción con asistente): Como adulto mayor, quiero presionar un botón de micrófono y preguntar con mi voz "¿A qué hora me toca mi próxima pastilla?" para obtener una respuesta auditiva sin tener que navegar por menús complejos
- HU-05 (Seguridad y emergencias): Como adulto mayor, quiero tener un Botón SOS siempre visible en mi pantalla principal para llamar inmediatamente a mi red de apoyo en caso de una urgencia.
- HU-06 (Alertas escaladas): Como cuidador, quiero recibir una notificación push en mi teléfono si el adulto mayor no confirma la toma de un medicamento crítico en un lapso de 15 minutos, para poder contactarlo preventivamente.

Documento de requerimientos

El documento formal de requerimientos ha sido estructurado bajo el estándar IEEE Std 830-1998 (Especificación de Requisitos de Software). Dicho documento (detallado en los Capítulos I, II y III del proyecto general) funge como el contrato técnico del sistema, englobando 18 requerimientos funcionales (RF) divididos en 5 módulos lógicos (Usuarios, Rutina, Interfaz, Asistente Lógico y Seguridad) y 6 requerimientos no funcionales (RNF).

Acumulado del producto (Product backlog en la herramienta de Gestión de Proyectos de Desarrollo de Software)

En la herramienta de Gestión de Proyectos, el Product Backlog se ha priorizado en los siguientes Sprints o fases de desarrollo:

1. Fase 1 (Base de Datos y Auth): Diseño relacional, creación de API REST, registro de usuarios, encriptación de contraseñas (bcrypt) y manejo de tokens (JWT).
2. Fase 2 (Core Business & UI): Interfaz Mobile-First (React Native), formularios de agenda/medicamentos, modo de alto contraste y sincronización bidireccional entre perfiles.
3. Fase 3 (Inteligencia y Accesibilidad): Integración de APIs Speech-to-Text y Text-to-Speech, programación del Motor de Inferencia (Lógica Proposicional) y aplicación estricta de la Regla de Restricción Diagnóstica.
4. Fase 4 (Resiliencia y Notificaciones): Implementación de la caché local para modo offline, configuración de Cron Jobs para el Hard Delete (retención de 30 días) y servicio de notificaciones push para emergencias.

Atributos de calidad

- Usabilidad (Accesibilidad): Cumplimiento del estándar WCAG AA con contraste mínimo de 4.5:1, fuentes escalables dinámicamente y Touch Targets ampliados para motricidad fina reducida.
- Rendimiento: Tiempos de carga de interfaz inferiores a 3 segundos y tiempos de respuesta del motor de inferencia menores a 2 segundos bajo condiciones normales de red.
- Seguridad: Encriptación de datos sensibles (algoritmos de hash), comunicaciones bajo protocolos SSL/TLS (HTTPS) y persistencia de sesión segura mediante tokens aislados.
- Disponibilidad (Tolerancia a fallos): Operatividad offline parcial asegurada mediante caché local para la agenda diaria y contactos de emergencia.

Preguntas de negocio

El sistema LumaIA fue diseñado para responder y solucionar tecnológicamente las siguientes preguntas estratégicas:

1. ¿Cómo reducimos la brecha digital y la frustración tecnológica en adultos mayores (mayores de 60 años) al gestionar su salud diaria?

2. ¿De qué manera los familiares (cuidadores) pueden monitorear el cumplimiento de los tratamientos médicos sin invadir la autonomía del adulto mayor?

3. ¿Cómo mitigamos los errores operativos del usuario, como el borrado accidental de registros o la pérdida de conexión a internet en momentos críticos?

4. ¿Cómo garantizamos que el uso de Inteligencia Artificial (asistente conversacional) en el entorno de salud sea 100% seguro y no incurra en negligencias (diagnósticos automatizados)?

VII. PREREQUISITOS (descripción de los requerimientos mínimos de hardware o software para la instalación y puesta en funcionamiento del sistema)

Para garantizar el correcto despliegue del entorno de producción y el funcionamiento óptimo de la aplicación móvil "LumaIA", se establecen los siguientes requerimientos de hardware y software:

Requerimientos de hardware (Infraestructura del servidor)

Mínimos (Entorno de Desarrollo/Pruebas):

- CPU: 2 vCores (Procesador a 2.0 GHz o superior).
- Memoria RAM: 4 GB.
- Disco Duro**:** 50 GB SSD.

Recomendados (Entorno de Producción):

- CPU: 4 vCores.
- Memoria RAM: 8 GB (Para soportar la concurrencia de peticiones al motor lógico).
- Disco Duro: 100 GB SSD o superior (Para el almacenamiento de los esquemas médicos y logs).

Requerimientos de Hardware (Cliente - Dispositivo Móvil)

- Mínimos: Smartphone con procesador Quad-Core, 2 GB de RAM y al menos 150 MB de almacenamiento interno disponible (para la app y la caché local). Micrófono y altavoz funcionales.

Requerimientos Mínimos de Software (Sistemas y Servidores)

- Sistema Operativo de los Servidores: Linux Ubuntu Server 22.04 LTS o 24.04 LTS (o en su defecto, Debian 12), elegido por su estabilidad, seguridad nativa y compatibilidad con contenedores.
- Servidores de Bases de Datos:
    - PostgreSQL versión 15.x / 16.x (Recomendado por su integridad referencial y manejo avanzado de datos).
    - *(Alternativa admitida):* MySQL 8.0.
- Servidores de Aplicación (Backend):
    - Entorno de ejecución: Node.js (v20.x LTS).
    - Gestor de procesos: PM2 (para mantener el backend en ejecución continua y gestionar reinicios automáticos).
- Servidores Web (Proxy Inverso):
    - Nginx versión 1.24 o superior. Actuará como proxy inverso para redirigir el tráfico desde los puertos estándar hacia el servidor Node.js y gestionar los certificados SSL.
- Sistema Operativo Móvil (Cliente):
    - Android 10.0 o superior.
    - iOS 14.0 o superior.
- Navegadores Compatibles (Para administración y documentación de la API REST):
    - Google Chrome (v115+), Mozilla Firefox (v115+), Safari (v16+).

Lenguajes de Programación Utilizados

- TypeScript / JavaScript (ES6+): Lenguaje principal utilizado tanto en el desarrollo de la aplicación móvil (frontend) como en la lógica del servidor (backend).

- SQL: Lenguaje estructurado para la manipulación, consulta y creación de las bases de datos relacionales.

VIII. FRAMEWORKS Y ESTÁNDARES DE DESARROLLO UTILIZADOS

Librerías, Frameworks, Controladores o Plugins

- Frontend (Móvil): React Native (con Expo o React Native CLI) para el desarrollo de componentes nativos multiplataforma.
- Backend (API & Lógica): NestJS (framework progresivo sobre Node.js) o Express.js para estructurar los endpoints REST.
- Seguridad: jsonwebtoken (para generación de tokens JWT) y bcrypt (para el hash de contraseñas).
- Interacción por Voz: APIs nativas de Google Speech-to-Text / Apple Speech (accesibles mediante librerías de React Native como @react-native-voice/voice).

Estándares de Codificación y Patrones de Diseño

- Patrón Arquitectónico: Modelo Cliente-Servidor acoplado a una arquitectura basada en capas (Controladores, Servicios, Repositorios) en el backend.
- Clean Code y SOLID: Estándares de programación orientada a objetos aplicados en el backend para facilitar el mantenimiento y la escalabilidad.
- Linter y Formateo: Uso de ESLint y Prettier para mantener una sintaxis homogénea en todo el código fuente del equipo de desarrollo.

Estándares de Identidad Corporativa y Look & Feel

- Ergonomía Visual: Diseño "Mobile-First" alineado a las normativas WCAG nivel AA.
- Colorimetría: Paleta de colores de alto contraste (Ratio mínimo de 4.5:1). Fondos claros o neutros con botones de acción primarios (como el Botón SOS) en colores cálidos y saturados (ej. rojo o naranja de alerta).
- Tipografía: Fuentes Sans-Serif (como Roboto o San Francisco) adaptativas, sin remates, para

maximizar la legibilidad en pantallas pequeñas, soportando escalabilidad dinámica del sistema operativo.

- Navegación: Limitada a un solo nivel jerárquico. Botones expansivos (Touch Targets mínimos de 48x48 dp) para compensar la pérdida de motricidad fina.

Puertos de Comunicación

- Puerto 80 (TCP): Tráfico HTTP (Será redirigido automáticamente al 443).
- Puerto 443 (TCP): Tráfico HTTPS (Protocolo seguro para comunicación Cliente-Servidor).
- Puerto 5432 (TCP): Puerto por defecto para la comunicación interna de la base de datos PostgreSQL (o 3306 si se usa MySQL). (Bloqueado al exterior por seguridad).
- Puerto 3000 / 4000 (TCP): Puertos de escucha internos del servidor Node.js/NestJS.

Protocolos de Seguridad

- En Tránsito: TLS 1.2 / 1.3 mediante certificados SSL para cifrar toda petición HTTP (HTTPS).
- Autenticación: Protocolo de portador (Bearer) utilizando JSON Web Tokens (JWT) con tiempo de expiración definido.
- En Reposo: Algoritmo *bcrypt* para el cifrado unidireccional (salting y hashing) de contraseñas y pines de vinculación en la base de datos.

Interfaces con Otros Sistemas (Integraciones)

- Notificaciones Push: Interfaz con Firebase Cloud Messaging (FCM) para Android y Apple Push Notification service (APNs) para iOS, para enviar alertas críticas al Cuidador.
- Hardware Nativo: Interfaz con el hardware del dispositivo móvil para uso del motor Text-to-Speech (altavoz) y Speech-to-Text (micrófono).

IX. MODELO DE DATOS

- Diagrama de casos de uso

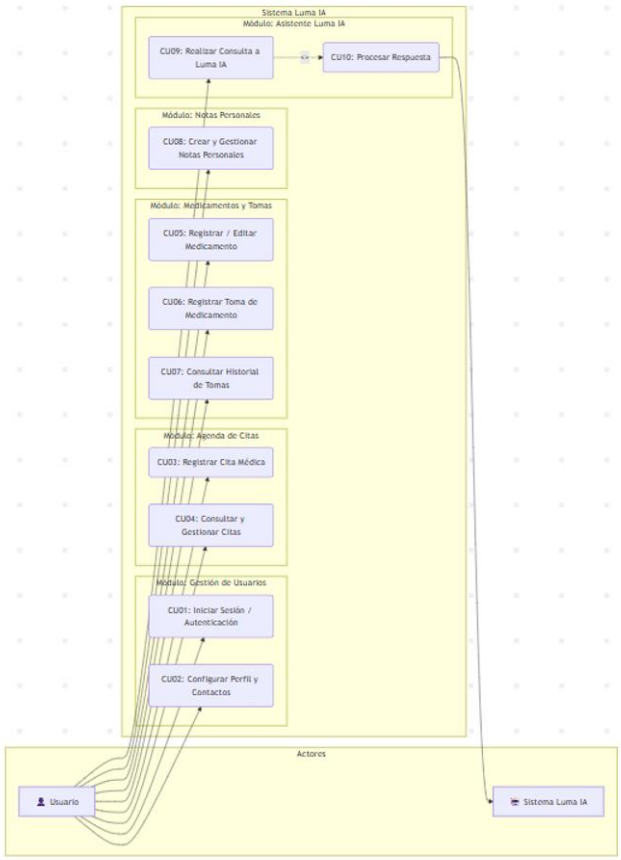


Ilustración 1 Diagrama de casos de uso

- Diagrama ER

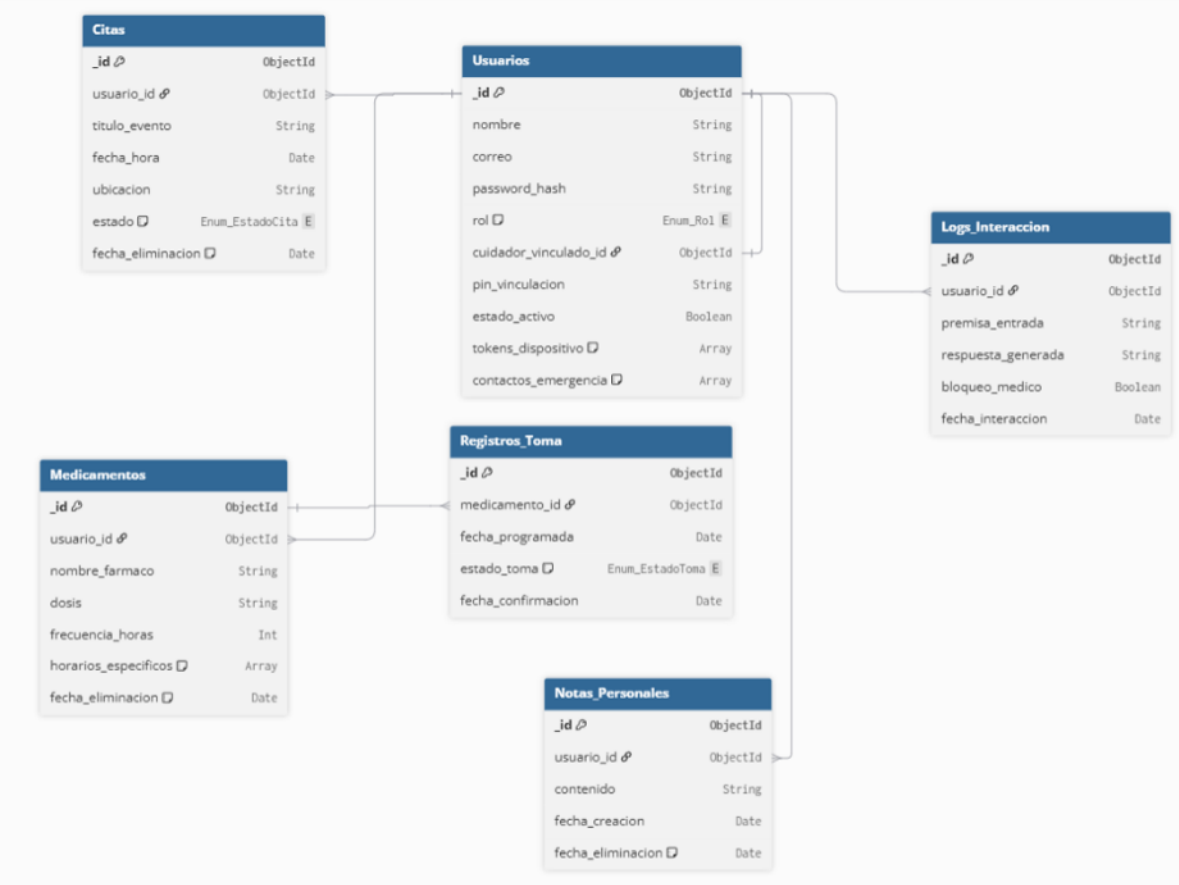


Ilustración 2 Diagrama entidad relación

- Diccionario de datos

Usuarios			
ObjectId	_id	PK	Identificador único del usuario
String	nombre		Nombre del usuario
String	correo		Correo electrónico
String	password_hash		Contraseña encriptada
Enum_Rol	rol		Rol asignado al usuario
ObjectId	cuidador_vinculado_id	FK	ID de referencia a otro usuario
String	pin_vinculacion		PIN o código de vinculación
Boolean	estado_activo		Estado de la cuenta (activo/inactivo)
Array	tokens_dispositivo		Tokens para notificaciones push
Array	contactos_emergencia		Lista de contactos de emergencia

Citas			
ObjectId	_id	PK	Identificador único de la cita
ObjectId	usuario_id	FK	Referencia al usuario (Usuarios._id)
String	titulo_evento		Título o asunto de la cita
Date	fecha_hora		Fecha y hora programada
String	ubicacion		Ubicación del evento o cita
Enum_EstadoCita	estado		Estado actual de la cita
Date	fecha_eliminacion		Fecha de borrado lógico

Medicamentos			
ObjectId	_id	PK	Identificador único del medicamento
ObjectId	usuario_id	FK	Referencia al usuario (Usuarios._id)
String	nombre_farmaco		Nombre del fármaco o medicina
String	dosis		Dosis especificada
Int	frecuencia_horas		Intervalo de toma en horas
Array	horarios_especificos		Horarios fijados para el consumo
Date	fecha_eliminacion		Fecha de borrado lógico

Registros_Toma			
ObjectId	_id	PK	Identificador único de la toma
ObjectId	medicamento_id	FK	Referencia al medicamento (Medicamentos._id)
Date	fecha_programada		Fecha y hora en que corresponde tomarlo
Enum_EstadoToma	estado_toma		Estado de la toma (tomada/omitida/etc.)
Date	fecha_confirmacion		Fecha y hora real de confirmación

Notas_Personales			
ObjectId	_id	PK	Identificador único de la nota
ObjectId	usuario_id	FK	Referencia al usuario (Usuarios._id)
String	contenido		Texto o cuerpo de la nota
Date	fecha_creacion		Fecha y hora de creación
Date	fecha_eliminacion		Fecha de borrado lógico

Logs_Interaccion			
ObjectId	_id	PK	Identificador único del registro de chat
ObjectId	usuario_id	FK	Referencia al usuario (Usuarios._id)
String	premisa_entrada		Mensaje/Prompt enviado por el usuario
String	respuesta_generada		Respuesta proporcionada por la IA
Boolean	bloqueo_medico		Bandera de bloqueo por seguridad médica
Date	fecha_interaccion		Fecha y hora de la interacción

Ilustración 3 Diccionario de datos

- Diagrama de clases

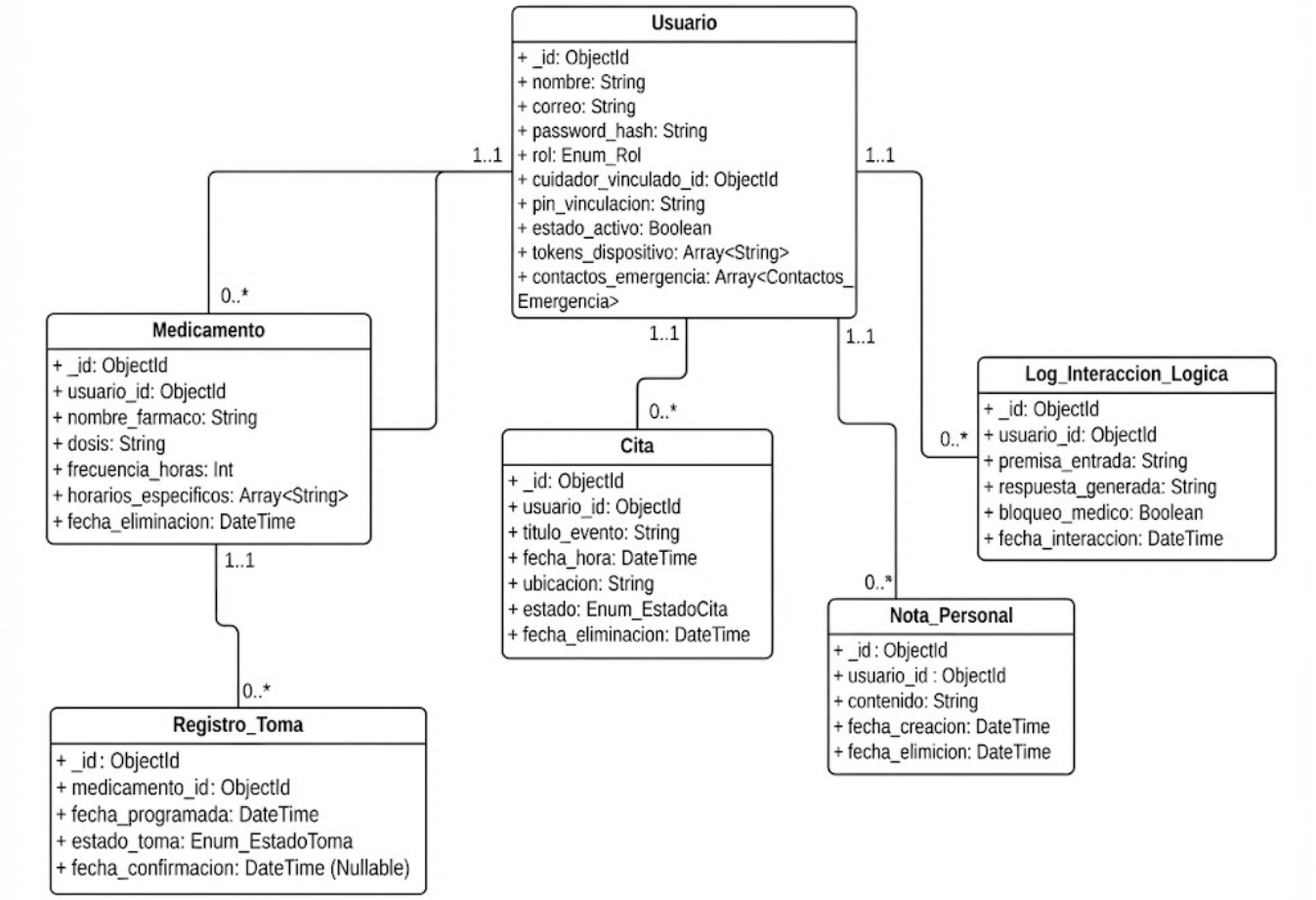


Ilustración 4 Diagrama de clases

X. FUNCIONALIDAD Y SERVICIOS OFRECIDOS

LumaIA se expone a través de una API RESTful centralizada que brinda los siguientes servicios operativos:

1. Servicio de autenticación y seguridad (Auth): Emisión y validación de tokens JWT, asegurando que solo usuarios y cuidadores vinculados accedan a la información médica.
2. Servicio de gestión de rutina (CRUD Médico): Creación, lectura, actualización y borrado lógico (Soft Delete) de medicamentos, citas y notas, con sincronización en tiempo real.
3. Servicio de motor lógico y conversacional: Procesamiento de cadenas de texto (derivadas de la voz del usuario) mediante lógica proposicional para devolver consultas de la agenda, bloqueando por defecto cualquier intento de diagnóstico (Regla RD-01).
4. Servicio de alertas y escalamiento (Push & SOS): Disparo automatizado de notificaciones al cuidador mediante Firebase/APNs tras la omisión de un evento crítico o la pulsación del Botón SOS.

XI. DESPLIEGUE Y CONFIGURACIÓN DE LOS COMPONENTES QUE CONFORMAN EL SISTEMA DE INFORMACIÓN

1. Organización de Componentes

El código fuente se administra a través de un sistema de control de versiones (Git), estructurado en dos repositorios principales para mantener la independencia (Desacoplamiento):

- Repositorio Frontend: Contiene el proyecto en React Native (Expo o CLI).
- Repositorio Backend: Contiene la lógica en Node.js/NestJS y los modelos de base de datos.

2. Instalación

- Base de Datos: Ejecutar los scripts DDL en PostgreSQL para crear las tablas, relaciones y extensiones (UUID).
- Backend: Ejecutar npm install o yarn install en el directorio raíz del servidor para descargar las dependencias (Express, bcrypt, jsonwebtoken, pg, etc.).
- Frontend: Instalar dependencias en el proyecto React Native. Si se usa Expo, ejecutar npx expo start para inicializar el servidor de desarrollo.

3. Configuración

El sistema depende de un archivo de variables de entorno (.env) en el backend. Las variables críticas que deben configurarse antes del despliegue son:

- PORT=3000 (Puerto de escucha local).
- DB_HOST, DB_USER, DB_PASS, DB_NAME (Credenciales seguras de PostgreSQL).
- JWT_SECRET (Clave secreta fuerte para encriptar los tokens de sesión).
- FCM_SERVER_KEY (Llave de servidor de Firebase para envío de notificaciones push).

4. Despliegue (Entorno de Producción)

- Backend: Utilizar PM2 (pm2 start dist/main.js --name "lumaia-api") para asegurar que el servicio se ejecute en segundo plano y se reinicie automáticamente ante fallas.
- Proxy Inverso: Configurar Nginx para interceptar el tráfico HTTPS (Puerto 443), aplicar el certificado SSL (Let's Encrypt) y redirigir internamente las peticiones al puerto 3000 de Node.js.
- Frontend (App): Generar los compilados finales: un archivo .aab o .apk para la distribución en Android, y el archivo .ipa compilado vía Xcode para iOS.

XII. RESOLUCIÓN DE PROBLEMAS

Error / Síntoma	Causa Probable	Solución Técnica
ERR-01: El motor lógico no escucha o no responde.	Permisos del dispositivo denegados o librería STT/TTS mal enlazada.	1. Verificar que la app tiene permisos explícitos de "Micrófono". 2. Reconstruir dependencias nativas en React Native (npx pod-install ios / gradlew clean en Android).
ERR-02: Connection Refused en el Backend al iniciar sesión.	El backend no puede conectar a la base de datos PostgreSQL.	Verificar que el servicio postgresql esté corriendo (systemctl status postgresql). Validar que las credenciales en el archivo .env coinciden y que el puerto 5432 no esté bloqueado por el firewall local (UFW).
ERR-03: Las notificaciones Push no llegan al Cuidador.	Certificados de Firebase (FCM) o APNs expirados o desconfigurados.	Renovar las credenciales desde la consola de Firebase o el panel de Apple Developer. Reiniciar el servicio PM2 para cargar las nuevas llaves del archivo .env.
ERR-04: Errores	Nginx o el framework	Configurar la política de CORS en el archivo

CORS al consultar la API desde un navegador.	backend no tienen habilitados los encabezados de *Cross-Origin*.	principal de Node.js, o añadir add_header Access-Control-Allow-Origin *; en la configuración del proxy Nginx.
ERR-05: El registro no desaparece tras pulsar "Eliminar".	Confusión entre *Soft Delete* y *Hard Delete*.	Comportamiento esperado. El registro cambió su estado a deleted_at = TIMESTAMP, pero no será borrado físicamente hasta que el Cron Job mensual del servidor lo purgue (RD-02).

XIII. REFERENCIAS

IEEE Computer Society. (1998). IEEE Recommended Practice for Software Requirements Specifications (IEEE Std 830-1998).

XIV. CONCLUSIONES

El diseño arquitectónico y lógico estructurado en este Manual Técnico demuestra que LumaIA es un sistema altamente viable, escalable y robusto. La elección tecnológica de React Native garantiza el cumplimiento del enfoque Mobile-First, fundamental para la accesibilidad visual y motriz del Adulto Mayor. Por su parte, la arquitectura del backend (Node.js + PostgreSQL) acoplada a directivas de negocio estrictas (como el Soft Delete a 30 días y la prohibición diagnóstica del Motor Lógico), asegura la integridad de los datos, la privacidad médica y el respeto a la autonomía del usuario, resolviendo la problemática base descrita en el modelado inicial.